

Memoria en C/C++

Facultad de Ingeniería
Universidad de Buenos Aires

Exacta reserva de memoria

```
char c = 'A';  
int i = 1;  
short int s = 4;  
char *p = 0;  
int *g = 0;  
int b[2] = {1, 2};  
char a[] = "AB";
```

- Todo depende de la arquitectura y del compilador
- Alineación y padding
- Punteros del mismo tamaño
- Un cero como "fin de string"

c	65			
i	0	0	0	1
s	0	4		
p	0	0	0	0
g	0	0	0	0
b	0	0	0	1
	0	0	0	2
a	65	66	0	

Agrupación de variables

```
struct S {  
    int a;  
    char b;  
    int c;  
    char d;  
};
```

```
struct S s = {1,2,3,4};
```

s.a	0	0	0	1
s.b	2			
s.c	0	0	0	3
s.d	4			

Agrupación de variables

```
struct S {  
    int a;  
    char b;  
    int c;  
    char d;  
} __attribute__((packed));
```

```
struct S s = {1,2,3,4};
```

s.a	0	0	0	1
s.b/s.c	2	0	0	0
s.c/s.d	3	4		

Endianess: representación en memoria

`int i = 1;`

`((unsigned char*)&i)`
`== {0, 0, 0, 1}`

0	0	0	1
---	---	---	---

big

`((unsigned char*)&i)`
`== {1, 0, 0, 0}`

1	0	0	0
---	---	---	---

little

`int j = 1023;`

`((unsigned char*)&j)`
`== {0, 0, 3, 255}`

0	0	3	ff
---	---	---	----

big

`((unsigned char*)&j)`
`== {255, 3, 0, 0}`

ff	3	0	0
----	---	---	---

little

Endianess: representación en memoria

Se puede cambiar el endianess de una variable **short int** y **int** del endianess nativo o "del host" a big endian o "el endianess de la red" y viceversa:

Host to Network

htons(short int) **htonl(int)**

Network to Host

ntohs(short int) **ntohl(int)**

Segmentos de memoria

- Code segment: de solo lectura y ejecutable, a donde va el código y las constantes.
- Data segment: variables creadas al inicio del programa y son válidas hasta que este termina; pueden ser de acceso global o local.
- Stack: variables creadas al inicio de una llamada a una función y destruidas automáticamente cuando está llamada termina.
- Heap: variables cuya duración está controlada por el programador (run-time).

Duración y visibilidad (lifetime and scope)

- Duración (lifetime): tiempo desde que a la variable se le reserva memoria hasta que esta es liberada.
Determinado por el segmento de memoria que se usa.
- Visibilidad (scope): Cuando una variable se la puede acceder y cuando esta oculta.

Asignación del lifetime y scope

```
int g = 1;           // Data segment; scope global
static int l = 1;    // Data segment; scope local (este file)
extern char e;        // No asigna memoria (es un nombre)

void Fa() { }         // Code segment; scope global
static void Fb() { }  // Code segment; scope local (este file)
void Fc();             // No asigna memoria (es un nombre)

void foo(int arg) {   // Argumentos y retornos son del stack
    int a = 1;         // Stack segment; scope local (func foo)
    static int b = 1;  // Data segment; scope local (func foo)
    void * p = malloc(4); // p en el Stack; apunta al Heap
    free(p);           // liberar el bloque explicitamente!!
    char *c = "ABC";   // c en el Stack; apunta al Code Segment
    char ar[] = "ABC"; // es un array con su todo en el Stack
} // fin del scope de foo: las variables locales son liberadas
```

El donde importa!

```
void f() {  
    char *a = "ABC";  
    char b[] = "ABC";  
  
    b[0] = 'X';  
    a[0] = 'X';    // segmentation fault  
}
```

Punteros

```
int *p;    // p es un puntero a int
           // (p guarda la dirección de un int)
int i = 1;
p = &i;    // &i es la dirección de la variable i

*p = 2;    // *p de-referencia o accede a la memoria
           // cuya dirección está guardada en p
/* i == 2 */

char buf[512];
write(&buf[0], 512);
```

Aritmética de punteros

```
int a[10];
```

```
int *p;
```

```
p = &a[0];
```

```
*p      // a[0]
```

```
*(p+1)  // a[1]
```

```
int *p;
```

```
p+1      // movete sizeof(int) bytes (4)
```

```
char *c;
```

```
c+2      // movete 2*sizeof(char) bytes (2)
```

Punteros a funciones (al code segment)

```
int g(char) {}
```

```
int (*p)(char);  
p = &g;
```

```
#include <stdlib.h>
```

```
void qsort( void *base,  
            size_t nmemb,  
            size_t size,  
            int (*cmp)(const void *, const void *))
```

```
);
```

```
int cmp_personas(const void* a, const void* b) {  
    struct Persona *pa = (struct Persona*)a;  
    struct Persona *pb = (struct Persona*)b;  
    return pa->edad - pb->edad;  
}
```

Como leer la notación de punteros en C/C++

/* Ejemplo 1 */

char *a[10];

a // "a"

***a // "a" apunta a**

char *a // "a" apunta a char

char *a[10]; // "a" apunta a char (10 de esos)

**char *a[10]; // "a" es un array de 10 de esos, o sea
 // "a" es un array de 10 punteros a char**

Como leer la notación de punteros en C/C++

/* Ejemplo 2 */

char (*c)[10];

c

// "c"

***c**

// "c" apunta a

(*c) == X

// llamemos "X" a (*c) temporalmente

char X[10];

char X[10];

// "X" es un char (10 de esos)

char X[10];

// "X" es un array de 10 char

char (*c)[10];

// "c" apunta a un array de 10 char

Como leer la notación de punteros en C/C++

/ Ejemplo 3: modo dios */*

char (*f)(int)[10];

f // "f"

*f // "f" apunta a

(*f) == X

char X(int)[10];

char X(int) // es la firma de una funcion,

// asi que vuelvo un paso para atras

char (*f)(int) // entonces esto es un puntero a funcion

// cuya firma recibe un int y retorna un char

char (*f)(int)[10]; // puntero a funcion, 10 de esos

char (*f)(int)[10]; // f es un array de 10 punteros a funcion,

// que reciben un int y retornan un chars

Simplificando la notación

```
char *X[10];           // la variable "X" es un array de  
                        // 10 punteros a char
```

```
typedef char *X[10];    // el tipo "X" es un array de  
                        // 10 punteros a char
```

```
X my_array;            // es una alias, decir "X" es como decir  
char *my_array[10];    // "array de 10 punteros a char"
```

Simplificando la notación

```
void (*X)();    // la variable "X" es un puntero a  
                // funcion
```

```
typedef void (*X)();    // el tipo "X" es un puntero a  
                        // funcion
```

```
X f[10];        // f es una array de 10 X, entonces  
                // f es una array de 10 punteros  
                // a funcion
```

Smash the stack for fun and profit

```
// compilar con el flag -fno-stack-protector
```

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {
```

```
    int cookie = 0;
```

```
    char buf[10];
```

```
    printf("buf: %08x cookie: %08x\n", buf, &cookie);
```

```
    gets(buf);
```

```
    if (cookie == 0x41424344) {
```

```
        printf("You win!\n");
```

```
    }
```

```
    return 0;
```

```
} // Insecure Programming
```

Buffer overflow

Funciones inseguras que no ponen un límite en el tamaño del buffer que usan. No usarlas!

```
gets(buf);  
strcpy(dst, src);
```

Reemplazarlas por funciones que sí permiten definir un límite, pero es responsabilidad del programador poner un valor coherente!

```
getline(buf, max_buf_size, stream);  
strncpy(dst, src, max_dst_size);
```

Challenge: hacer que el programa imprima "You win!"

```
// compilar con el flag -fno-stack-protector
```

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {
```

```
    int cookie = 0;
```

```
    char buf[10];
```

```
    printf("buf: %08x cookie: %08x\n", buf, &cookie);
```

```
    gets(buf);
```

```
    if (cookie == 0x41424344) {
```

```
        printf("You loose!\n");
```

```
    }
```

```
    return 0;
```

```
} // Insecure Programming
```